

KOSAC Lexicon for Social Scientists

2024-01-18 Sumin

배경

KOSAC(KOrean Sentiment Analysis Corpus)

- <https://knlp.snu.ac.kr/kosac>
- 감정분석 코퍼스: Hayeon Jang, Munhyong Kim, and Hyopil Shin. 2013. KOSAC: A Full-Fledged Korean Sentiment Analysis Corpus. In *Proceedings of the 27th Pacific Asia Conference on Language, Information, and Computation (PACLIC 27)*, pages 366–373, Taipei, Taiwan. Department of English, National Chengchi University.
- 감정사전: 박수민 (2016)

KOSAC 감정사전의 최근 활용 사례

- 분야: 사회과학 연구
- 기능: 텍스트의 긍정/부정 분류
- 방법: 단순 빈도 계산
- 예시
 - 노예영, 김기한. (2023). 올림픽 스포츠 콘텐츠의 교량적 사회자본 형성 효과 : 다차원적 네트워크 구조 분석을 중심으로. 한국스포츠산업경영학회지, 28(2), 1-23, 10.31308/KSSM.28.2.1

트윗의 논조는 긍정/부정/양가적으로 나누었으며 이는 앞서 수집한 트윗별 형태소를 바탕으로 한국어 감성 분석 코퍼스인 KOSAC(Korean Sentiment Analysis Corpus, Shin et al., 2012)을 사용했다. ... 즉 **KOSAC 사전에 포함된 단어를 바탕으로 트윗의 단어들을 분류하고, 항목별(긍정/부정/양가) 단어 수 및 총 단어 수 대비 점유율의 형태로 결과 값을 산출했다.**

- 여현진. (2022). 텍스트마이닝을 통한 날씨정보와 감성변화 예측모형 개발. 한국컴퓨터정보학회논문지, 27(8), 219-224.

본 연구는 **KOSAC의 게시물 내 말뭉치의 긍정/부정 빈도에 따라 게시물 자체의 긍정/부정을 분류하였으며**, 게시물들의 긍정/부정 판단은 전체 게시물들의 긍정 말뭉치 빈도와 부정 말뭉치 빈도의 차이의 절대 값 평균을 기준으로 게시물들의 긍정/부정을 결정하였으며, 차이가 평균치를 밑도는 경우 중립글로 판별하여 본 분석에 사용하지 않았다.

- 한계 지적
 - 신다은, 오하영. (2023). 우울증 진단 및 치료를 위한 챗봇 연구 동향 분석. 한국정보통신학회논문지, 27(3), 337-347, 10.6109/jkiice.2023.27.3.337

KOSAC은 하나의 단일 형태소로만 구성된 것이 아니라 **여러 가지 형태소가 섞여 있는** 절도 많이 포함하고 있다. 이런 경우, 조사까지 함께 포함된 경우가 많은데, KoELECTRA로 토큰화를 진행할 때는 조사가 따로 분리되는 점을 고려하면 해당 데이터들은 **KoELECTRA를 이용한**

감성 사실 보낼 줄 만드는 네에 적절하지 않다. 따라서 단일 형태소로 구성된 데이터만을 사용하기로 하고, 단일 형태소를 제외하고 조사를 포함한 그 외 데이터를 삭제하기로 한다.

추가로 단어도 같고 형태소도 같아 **중복되는 단어들이 존재하였는데**, 이 경우에는 빈도수가 높은 단어 하나만을 남겨두고 나머지 중복 단어는 제거한다.

또한, KOSAC 감성 사전은 하나의 문장을 가져온 후 해당 문장을 단어 별로 쪼개 단어마다 감성 값을 부여하는 방식으로 만들어졌다. 예를 들어 ‘불행’이라는 단어는 ‘불행감에서 벗어나’라는 문장에서 쪼개져 나온 단어다. 이러한 이유로 긍정인지 부정인지 잘못 분류된 단어들이 있으며, 그 대표적인 예시가 ‘불행’이라는 단어이다. ‘불행감에서 벗어나’라는 문장이 긍정이기 때문에 해당 문장에서 파생된 항목들인 ‘불행’, ‘불행감’, ‘불행감에서’ 모두 긍정으로 분류된 것을 확인할 수 있었다.

계획

1. 사회과학 분야에서 KOSAC을 더 효율적으로 사용할 수 있도록 파이썬 라이브러리를 개발한다.
2. 기존에 제기된 문제점을 해결한다.

기본 재료

1. 기존 감성사전 (표제어: 형태소 n-gram)

기본 파이프라인

1. 한국어 문장을 입력받는다.
2. POS tagger를 사용하여 형태소로 분할한다.
 - 옵션1: 세종 태그셋을 쓰는 konlpy.tag.Komorani를 이용한다.
 - 옵션2: 유사한 태그셋을 쓰는 konlpy.tag.Kkma를 이용한다.
3. 형태소 시퀀스를 감성사전과 대조하여 감성단어를 추출한다.
 - 옵션1: 확률이 높은 표제어부터 매치시킨다.
 - 옵션2: 3-gram, 2-gram, 1-gram의 순서로 긴 것부터 매치시킨다.
4. 문장 내 감성단어의 목록을 출력한다.
5. 감성단어 각각의 확률을 사용하여 문장 전체의 긍정/부정 확률을 계산한다.

부가 기능

1. 1-gram, 2-gram, 3-gram 중에서 원하는 범위를 선택할 수 있게 만든다.
2. 품사 목록 중 원하는 범위를 선택할 수 있게 만든다(NNG, XR, VA, VV 등).
3. 다양한 smoothing 방법을 제시한다.

확장 기능

1. 기존 사전에 표제어를 추가한다.

2. 감정표현(seed)마다 긍정/부정 레이블이 부착된 코퍼스로부터 감정사전을 자동으로 도출한
데모

```
In [ ]: from utils import *
```

SentLex 클래스: (일반화 가능한) 감정사전

기존 감정사전 초기화하기

expressive-type, intensity, nested-order, **polarity**, subjectivity-polarity, subjectivity-type

```
In [ ]: # raw polarity lexicon
kosac = SentLex()
kosac.original_lexicon
```

```
Out[ ]:
```

	ngram	freq	COMP	NEG	NEUT	None	POS	max.value	max.prop
entry									

ㄴ/ETM	1	289	6	107	16	21	137	POS	0.474048
ㄴ/ETM 가슴/NNG	2	1	0	0	0	0	1	POS	1.000000
ㄴ/ETM 가슴/NNG 은 /JX	3	1	0	0	0	0	1	POS	1.000000
ㄴ/ETM 감/NNG	2	1	0	1	0	0	0	NEG	1.000000
ㄴ/ETM 감/NNG 이 /JKS	3	1	0	1	0	0	0	NEG	1.000000
...	...	...	...	...	...	...	...	...	...
힘들/VA 고/EC	2	1	0	0	0	0	1	POS	1.000000
힘들/VA 고/EC 외롭 /VA	3	1	0	0	0	0	1	POS	1.000000
힘차/VA	1	1	0	1	0	0	0	NEG	1.000000
힘차/VA ㄴ/ETM	2	1	0	1	0	0	0	NEG	1.000000
힘차/VA ㄴ/ETM 붓 /NNG	3	1	0	1	0	0	0	NEG	1.000000

16192 rows × 9 columns

표제어 필터링

KOSAC 코퍼스 내 출현 횟수의 최솟값을 설정할 수 있다.

```
In [ ]: # 5회 이상 출현한 형태소 n-gram
kosac.set_lexicon(min_freq=5)
kosac.get_lexicon()
```

Out[]:

	ngram	freq	COMP	NEG	NEUT	None	POS	max.value	max.prop
entry									
L/ETM	1	289	6	107	16	21	137	POS	0.474048
L/ETM 것/NNB	2	29	0	6	3	9	8	None	0.344828
L/ETM 것/NNB 같 /VA	3	5	0	2	0	2	1	NEG	0.400000
L/ETM 것/NNB 이 /JKS	3	5	0	3	1	0	1	NEG	0.600000
L/ETM 것/NNB 이 /VCP	3	9	0	0	1	7	0	None	0.777778
...	...	...	...	...	...	...	...	...	...
확인/NNG	1	5	0	4	0	0	1	NEG	0.800000
훨씬/MAG	1	9	0	2	0	0	6	POS	0.666667
흐르/VV	1	5	0	4	0	0	1	NEG	0.800000
흥행/NNG	1	6	0	0	0	0	6	POS	1.000000
힘/NNG	1	7	0	1	0	0	5	POS	0.857143

613 rows × 9 columns

```
In [ ]: kosac.set_lexicon(min_freq=1)
        kosac.get_lexicon()
```

```
Out[ ]:
```

	ngram	freq	COMP	NEG	NEUT	None	POS	max.value	max.prop
entry									
ㄴ/ETM	1	289	6	107	16	21	137	POS	0.474048
ㄴ/ETM 가슴/NNG	2	1	0	0	0	0	1	POS	1.000000
ㄴ/ETM 가슴/NNG 은 /JX	3	1	0	0	0	0	1	POS	1.000000
ㄴ/ETM 감/NNG	2	1	0	1	0	0	0	NEG	1.000000
ㄴ/ETM 감/NNG 이 /JKS	3	1	0	1	0	0	0	NEG	1.000000
...	...	...	...	...	...	...	...	...	...
힘들/VA 고/EC	2	1	0	0	0	0	1	POS	1.000000
힘들/VA 고/EC 외롭 /VA	3	1	0	0	0	0	1	POS	1.000000
힘차/VA	1	1	0	1	0	0	0	NEG	1.000000
힘차/VA ㄴ/ETM	2	1	0	1	0	0	0	NEG	1.000000
힘차/VA ㄴ/ETM 붓 /NNG	3	1	0	1	0	0	0	NEG	1.000000

16192 rows × 9 columns

사전에 토큰 추가하기 (1) 기존 용례 추가

새로운 데이터에서 '귀엽/VA'이라는 형태소가 긍정적으로 쓰인 표현이 1개 있다고 가정하자.

```
In [ ]: # 기존 사전의 '귀엽/VA' 정보: KOSAC 코퍼스에서는 부정적으로 1회 사용되었다.
        morph = '귀엽/VA'
        kosac.get(morph)
```

```
Out[ ]: ngram      1
        freq       1
        COMP       0
        NEG        1
        NEUT       0
        None       0
        POS        0
        max.value   NEG
        max.prop    1.0
        Name: 귀엽/VA, dtype: object
```

```
In [ ]: # 토큰 추가: 새로운 데이터에서 '귀엽/VA'이 긍정적으로 쓰인 사례 1건을 기존 데이터에 추가함
        kosac.add_token(*('귀엽/VA', 'POS'), verbose=True)
```

```

ngram          1
freq           2
COMP           0
NEG            1
NEUT           0
None           0
POS            1
max.value      NEG
max.prop       0.5
Name: 귀엽/VA, dtype: object

```

SentLex.add_token() 메소드로 토큰을 추가하면 max.value 속성값과 max.prop 속성값이 자동으로 업데이트된다.

사전에 토큰 추가하기 (2) 새로운 엔트리 추가

몇 년 전의 신조어로 '커엽/VA'이라는 형태소가 있다(야민정음).

```
In [ ]: # 기존 사전의 정보: KOSAC 사전에는 당연히 '커엽/VA'이 없다.
        '커엽/VA' in kosac.lexicon.index
```

```
Out[ ]: False
```

```
In [ ]: # 토큰 추가: '커엽/VA'이 긍정적으로 쓰인 사례 1건을 기존 데이터에 추가한다.
        kosac.add_token(*('커엽/VA', 'POS'), verbose=True)
```

```

ngram          0
freq           1
COMP           0
NEG            0
NEUT           0
None           0
POS            1
max.value      POS
max.prop       1.0
Name: 커엽/VA, dtype: object

```

```

/Users/t/Documents/kosac-lexicon-private/utils.py:81: RuntimeWarning: invalid value encountered in long_scalars
  row['max.prop'] = counts.max() / counts.sum()

```

사전에 토큰 추가하기 (3) 다수의 용례 추가

```
In [ ]: examples = [
        ('귀엽/VA', 'POS'),
        ('귀엽/VA', 'POS'),
        ('귀엽/VA', 'NEG'),
        ('커엽/VA', 'POS'),
        ]
```

```
In [ ]: kosac.update(examples)
```

```
In [ ]: kosac.get('귀엽/VA')
```

```
Out[ ]: ngram          1
        freq          5
        COMP          0
        NEG           2
        NEUT           0
        None           0
        POS            3
        max.value      POS
        max.prop       0.6
        Name: 귀엽/VA, dtype: object
```

```
In [ ]: kosac.set_lexicon(min_freq=5)
```

Tagger 클래스: 형태소 분석기

```
In [ ]: # Use Komoran Tagger
        komoran = Tagger()
```

문장을 형태소 단위로 분리하기

세종 태그셋을 완전하게 지원하는 Komoran 분석기를 사용한다.

```
In [ ]: # morpheme tokenizer
        sentence = '모든 인간은 태어날 때부터 자유로우며 그 존엄과 권리에 있어 동등하다.'
        komoran.tokenize(sentence)
```

```
Out[ ]: '모든/MM 인간/NNG 은/JX 태어나/VV ㄹ/ETM 때/NNG 부터/JX 자유/NNG 례/XSA 며/EC 그/
        MM 존엄/NNG 과/JKB 권/NNG 리/NNB 에/JKB 있/VV 어/EC 등/NNG 등/NNB 하/XSA 다/EF
        ./SF'
```

문장에서 감정사전 표제어 찾기

주의: 바이그램과 트라이그램을 사용하는 경우, 감정사전 표제어의 정렬 방식에 따라 단어의 리스트가 달라질 수 있다.

```
In [ ]: print(kosac.match(sentence, komoran, sorting=True))
        print(kosac.match(sentence, komoran, sorting=False))

['모든/MM', '인간/NNG', '은/JX', '나/VV', 'ㄹ/ETM 때/NNG', '부터/JX', '자유/NN
G 례/XSA', '며/EC', '그/MM', '과/JKB', '에/JKB', '있/VV', '어/EC', '등/NNB',
'하/XSA', '다/EF']
['모든/MM', '인간/NNG', '은/JX', '나/VV', 'ㄹ/ETM', '때/NNG', '부터/JX', '자유
/NNG', '례/XSA', '며/EC', '그/MM', '과/JKB', '에/JKB', '있/VV', '어/EC', '등
/NNB', '하/XSA', '다/EF']
```

문장에 출현한 감정사전 표제어 세기

많은 사회과학 논문에서 사용하고 있는 방법...

```
In [ ]: # find sentiment lexicon items
        kosac.get_match_info(sentence, komoran)
```



```
Out[ ]: [('모든/MM', 'POS', 0.6),
        ('인간/NNG', 'NEG', 0.6),
        ('은/JX', 'NEG', 0.559322034),
        ('나/VV', 'POS', 0.714285714),
        ('르/ETM 때/NNG', 'NEUT', 0.428571429),
        ('부터/JX', 'POS', 0.625),
        ('자유/NNG 례/XSA', 'POS', 0.6),
        ('며/EC', 'NEG', 0.444444444),
        ('그/MM', 'POS', 0.538461538),
        ('과/JKB', 'POS', 0.636363636),
        ('에/JKB', 'POS', 0.378205128),
        ('있/VV', 'None', 0.455357143),
        ('어/EC', 'POS', 0.495867769),
        ('등/NNB', 'POS', 0.454545455),
        ('하/XSA', 'POS', 0.612612613),
        ('다/EF', 'NEG', 0.435483871)]
```

개선 가능성

1. 각 단어의 확률 정보를 활용할 수 있다.
2. 문장의 긍정/부정 확률을 제시할 수 있다.

문장의 감정 확률 계산

```
In [ ]: # Laplace smoothing
smooth(kosac.get('인간/NNG'))
```

```
Out[ ]: COMP    0.1
        NEG     0.4
        NEUT    0.1
        None    0.1
        POS     0.3
        Name: 인간/NNG, dtype: float64
```

```
In [ ]: kosac.get_smoothed_lexicon()
```

Out[]:		COMP	NEG	NEUT	None	POS
	entry					
	L/ETM	0.023973	0.369863	0.058219	0.075342	0.472603
	L/ETM 것/NNB	0.032258	0.225806	0.129032	0.322581	0.290323
	L/ETM 것/NNB 같/VA	0.100000	0.300000	0.100000	0.300000	0.200000
	L/ETM 것/NNB 이/JKS	0.100000	0.400000	0.200000	0.100000	0.200000
	L/ETM 것/NNB 이/VCP	0.076923	0.076923	0.153846	0.615385	0.076923
	...	...	...	...	...	...
	확인/NGG	0.100000	0.500000	0.100000	0.100000	0.200000
	월씬/MAG	0.076923	0.230769	0.076923	0.076923	0.538462
	흐르/VV	0.100000	0.500000	0.100000	0.100000	0.200000
	흥행/NGG	0.090909	0.090909	0.090909	0.090909	0.636364
	힘/NGG	0.090909	0.181818	0.090909	0.090909	0.545455

613 rows × 5 columns

```
In [ ]: kosac.get_sent_probs(sentence, komoran)
```

```
Out[ ]: POS      9.692229e-01
        NEG      3.077713e-02
        NEUT     1.368730e-10
        None     1.159872e-11
        COMP     9.217039e-15
        dtype: float64
```

```
In [ ]: kosac.get_sent_probs('학교중이 땡땡 댕 어서 모이자', komoran)
```

```
Out[ ]: NEG      0.250
        None     0.250
        POS      0.250
        COMP     0.125
        NEUT     0.125
        dtype: float64
```

```
In [ ]: kosac.get_sent_probs('빗물이 흐르고 내 눈물도 흐르고 잃어버린 첫사랑도 흐르네', komo)
```

```
Out[ ]: NEG      9.918663e-01
        POS      8.133718e-03
        None     1.642920e-08
        NEUT     8.682237e-10
        COMP     7.051358e-13
        dtype: float64
```